

INTERA Project Architecture & Technical Documentation

This document provides a comprehensive overview of how the **INTERA** Flutter application is built, the technologies it uses, the purpose of each file, and how data is transferred between the client app and the backend.

1. High-Level Technology Stack

The application is built on top of a modern mobile-first stack:

- * **Frontend Framework:** [Flutter \(Dart\)](#) — Multi-platform framework providing the client-side UI, interactions, state management, and device integration.
- * **Backend-as-a-Service (BaaS):** [Firebase](#) — Powering all serverless backend capabilities:
- * **Firebase Authentication:** Handles secure user registration, login, and token management.
- * **Cloud Firestore:** Real-time NoSQL database storing posts, comments, stories, direct messages, user profiles, and notifications.
- * **Firebase Storage:** Object storage for media assets (images, videos, profile pictures).
- * **Firebase Cloud Messaging (FCM):** Delivers push notifications to devices.
- * **Content Moderation: NSFW Detection Service** — Intercepts media uploads to ensure safety compliance.

2. Directory Structure & Core Modules

The codebase follows a feature-first architecture, separating concerns into discrete subfolders under `lib/`.

```
graph TD
    A["lib/main.dart"] --> B["lib/core"]
    A --> C["lib/features"]
    A --> D["lib/shared"]

    B --> B1["constants"]
    B --> B2["services"]
    B --> B3["routes"]
    B --> B4["theme"]

    C --> C1["auth"]
    C --> C2["home"]
    C --> C3["messaging"]
    C --> C4["profile"]
    C --> C5["videos"]
    C --> C6["notifications"]

    D --> D1["models"]
    D --> D2["widgets"]
```

■ lib/core/ (Application-wide singletons & configurations)

- `services/`

- [auth_service.dart](#): Wraps `FirebaseAuth` to manage user sessions (login, logout, sign-up, email verification).
- [feed_algorithm.dart](#): Implements sorting and scoring logic to rank posts for a user's home feed based on engagement (likes, comments, views, karma).
- [follow_service.dart](#): Handles follower/following transactions (subscribing to users, updating social counters).
- [notification_service.dart](#): Configures local system tray alert overlays, FCM tokens, and adds/streams notification alerts.
- [nsfw_detection_service.dart](#): Detects safe-for-work content using image processing APIs prior to publishing uploads.
- [permission_service.dart](#): Handles iOS/Android system permission requests (Camera, Gallery, Notifications).
- [reaction_service.dart](#): Facilitates adding reactions (Beauty, Art, Funny) to posts and awarding corresponding Karma points.
- `routes/`
- [app_routes.dart](#): Centralized application routing table defining screen paths (`/login`, `/home`, `/chat`, etc.) and transitions.
- `theme/`
- [app_theme.dart](#): Curated dark/light design tokens (gradients, colors, typography, borders, glassmorphism filters).

■ `lib/shared/` (Re-usable models and common widgets)

- `models/`
- [post_model.dart](#): Schema definition mapping Firestore documents into structured `Post` class objects.
- [user_model.dart](#): Schema mapping user profiles (`AppUser`).
- `widgets/`
- Contains common avatars, username tiles, buttons, and custom layout items.

■ `lib/features/` (UI Pages and local widgets grouped by feature)

- `auth/`: Splash screen, onboarding, sign-in, and registration screens.
- `home/`
- `home_feed_screen.dart`: The core landing feed showcasing global posts.
- `spark_viewer_screen.dart` / `create_spark_screen.dart`: The Instagram-style short-lived sparks (stories) player and composer.
- `messaging/`: Displays user inbox chats (`messages_list_screen.dart`) and manages real-time messaging layout interfaces (`chat_screen.dart`).
- `profile/`: Shows the user profiles, settings panel, and profile editor.

- `notifications/`: Renders notifications history (likes, comments, spark reactions).

3. Data Flow (Sending and Receiving)

All interactions follow a strict flow:

```
[User Action in UI]
  --> [Local State Updates (setState/Animation)]
    --> [Firestore Service Write / Firebase Storage Upload]
      --> [Cloud Backend / Database rules validation]
        --> [Real-time Stream / Listener notifications]
          --> [UI Updates automatically]
```

Here are specific execution pipelines for core activities:

A. Publishing a New Spark (Story)

```
sequenceDiagram
    autonumber
    actor User
    participant App as Flutter Client
    participant Storage as Firebase Storage
    participant DB as Firestore (stories)

    User->>App: Record/Upload media + enter caption
    App->>App: Check Safety (nsfw_detection_service)
    App->>Storage: Upload media file (mp4/jpg)
    Storage-->>App: Return public download URL
    App->>DB: Add document {authorId, mediaUrl, caption, viewedBy: [], createdAt, expiresAt}
    DB-->>App: Success
    App->>User: Close composer & play animation
```

B. Viewing a Spark (Story) & Tracking Viewers

1. **Receiving/Streaming:** The app queries stories created within the last 24 hours.
2. **Recording View:** When User B views User A's Spark, the client updates the document: `dart`
`Firestore.instance.collection('stories').doc(sparkId).update({ 'viewedBy':
 FieldValue.arrayUnion([currentUid]) }});`
3. **Retrieving Viewers:** When User A opens the viewers panel:
4. The app fetches the `viewedBy` array of UIDs from the spark document.
5. Performs an asynchronous loop querying `users/{uid}` for each viewer's name, avatar URL, and username to render the listing.

C. Sending a Reply to a Spark

1. **Publishing:** User B enters text and hits send. The client writes to a nested subcollection:

```
dart  
FirebaseFirestore.instance .collection('stories') .doc(sparkId) .collection('replies')  
    .add({'senderId': currentUid, 'text': text, 'createdAt': serverTimestamp()});
```
2. **Notification Dispatch:** The client writes to the global notifications collection:

```
dart  
NotificationService.sendNotification( recipientId: authorId, type: 'spark_reply', title:  
    '$senderName replied to your spark', subtitle: text, relatedId: sparkId, );
```
3. **Notification Pull:** User A's app, which has a real-time snapshot listener on the notifications collection, receives this insert and updates the unread notifications count badge on-screen immediately.

D. Real-Time Chat Messaging

1. **Listening (Receiving):** The chat screen subscribes to a Firestore snapshot listener:

```
dart  
FirebaseFirestore.instance .collection('conversations') .doc(conversationId)  
    .collection('messages') .orderBy('createdAt', descending: true) .snapshots()
```

 This ensures that whenever a new message is added to that subcollection, the UI rebuilds instantly to show it.
2. **Sending:** A user types a message and clicks send. The client:
3. Adds the document to the `messages` subcollection.
4. Updates the parent `conversations/{id}` document with the `lastMessageAt`, `lastMessageText`, and `unread` details.

4. Security & Access Control

Access to database records is governed by [firestore.rules](#). * **Reads:** Authenticated users can read most documents (posts, comments, sparks). * **Writes:** * Users can only edit/delete **their own** posts, comments, sparks, and profile documents. * **Interactions exception:** Users are allowed to perform partial updates to specific engagement fields on other users' documents (e.g. incrementing a post's `likeCount` or appending their ID to a story's `viewedBy` array). Everything else is rejected at the database level for safety.